Guide to Machine Learning in Pandora

Andy Chappell, Isobel Mawby, Matt Osbiston

October 23, 2026

Abstract

This document outlines the procedures for training and using deep neural networks within the Pandora pattern recognition framework.

1 Introduction

As of LArContent v04.00.00, Pandora has supported the use of neural networks via the LibTorch package. To date, a number of neural networks have found application in various reconstruction workflows across multiple LArTPC experiments. In particular, there are neural networks that handle neutrino interaction vertex identification, higher order vertex identification, separation of supernova neutrino interactions from low energy background interactions, and determination of the hierarchical relationships between reconstructed particles.

Each section of this document focuses on one such network and outlines the procedure for training and using that network.

2 Primary vertexing

Available since v04.00.00.

2.1 Training set preparation

Generation of the training samples is handled within the DlVertexingAlgorithm. The `PrepareTrainingSample` function generates a set of output CSV files for each of the views U, V and W. The XML files for training can be found in `settings/development` folder of the LArReco repository. In particular, `PandoraSettings_DLTrain_Pass1_Accel_DUNEFD.xml`, `PandoraSettings_DLTrain_Pass2_Accel_DUNEFD.xml`, `PandoraSettings_DLTrain_Pass1_Atmos_DUNEFD.xml`, `PandoraSettings_DLTrain_Pass2_Atmos_DUNEFD.xml`, `PandoraSettings_DLTrain_Pass1_LowE_DUNEFD_VD.xml` and `PandoraSettings_DLTrain_Pass2_LowE_DUNEFD_VD.xml`. At the time of writing, the training configuration is identical for long baseline (Accel) and atmospheric (Atmos) workflows. As the networks are currently dense U-ResNets, it is necessary to split the vertex identification into two passes to obtain the required reconstruction resolution. For pass 1 training, an example XML is

```
<algorithm type = "LArDLVertexing">
  <TrainingMode>true</TrainingMode>
  <TrainingOutputFileName>DUNEFD_MC11_Accel_Pass1</TrainingOutputFileName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames
      >
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875
      0.59125 0.67375 0.75625 0.85 1.0</DistanceThresholds>
</algorithm>
```

This will generate output comma-separated value (CSV) files called `DUNEFD_MC11_Accel_Pass1_CaloHitListU.csv`, `DUNEFD_MC11_Accel_Pass1_CaloHitListV.csv`, and `DUNEFD_MC11_Accel_Pass1_CaloHitListW.csv`. The distance thresholds define the boundaries of the distance bins into which pixels will be classified. The requirement here is for N + 1 boundaries (for N bins), defined on the interval

[0, 1]. This can be varied to suit your use case, as long as the distance thresholds are consistent between training and inference.

The training for low energy (available since v04.11.00) follows the same procedure as for the LBL and atmospheric cases, with a couple a minor differences. Given that smaller interactions dominate, there is less need for many classes to cover the range of distances, so this is reduced from 20 classes to 10 classes, with corresponding adjustments to the thresholds for class bins. For low energy pass 1 training, the thresholds are

```
<DistanceThresholds>0. 0.005 0.009 0.015 0.025 0.04 0.06 0.18 0.25 1.0</
    DistanceThresholds>
```

Returning to the LBL example, for pass 2 training, the XML structure is updated as follows

```
<algorithm type = "LArDLVertexing">
  <TrainingMode>false</TrainingMode>
  <OutputVertexListName>NeutrinoVertices3D_Pass1</OutputVertexListName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames
      >
  <ModelFileNameU>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_U_v04_03_00.pt</ModelFileNameU>
  <ModelFileNameV>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_V_v04_03_00.pt</ModelFileNameV>
  <ModelFileNameW>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_W_v04_03_00.pt</ModelFileNameW>
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875
      0.59125 0.67375 0.75625 0.85 1.0</DistanceThresholds>
</algorithm>

<algorithm type = "LArDLVertexing">
  <TrainingMode>true</TrainingMode>
  <Pass>2</Pass>
  <ImageWidth>128</ImageWidth>
  <ImageHeight>128</ImageHeight>
  <TrainingOutputFileName>DUNEFD_MC11_Accel_Pass2</TrainingOutputFileName>
  <InputVertexListName>NeutrinoVertices3D_Pass1</InputVertexListName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames
      >
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875
      0.59125 0.67375 0.75625 0.85 1.0</DistanceThresholds>
</algorithm>
```

The key differences here are the requirement to run the trained pass 1 network first, in inference, rather than training mode, provide the name of an output vertex list, and specify the names of the trained models from pass 1 for each of the respective views. This is important because training on a true vertex in pass 2 would cause the vertex to be located in preferred positions due to the framing of the image and potentially bias the network. Also notice that we specify the pass in the second instance, and also specify image dimensions in pass 2, which is half of the default dimension, for computational efficiency.

2.2 Training the network

Python Notebooks are provided for training the network. These notebooks are self-documenting, so only a brief overview will be given here. You will need the CSV files created during training set preparation, starting with the pass 1 files, given the dependence of pass 2 on pass 1 training. The `make_images.ipynb` notebook can be run from start to finish, modulo any edits that may be needed according to any customisations you applied (as described in the notebook).

The `network.ipynb` notebook can then run on the resultant compressed numpy arrays from the image generation step. You'll need GPU resources for this, and ensure your files are available on the GPU node (or a suitable high speed, highly parallel I/O system). As with the previous notebook,

potential edits according to customisation and which view you are running on will need to be made, but otherwise you can run the notebook from start to finish.

This will produce the trained networks for every epoch, and the TorchScript (.pt) network that will actually run in Pandora, along with various metrics showing training and validation loss and accuracy, to check for over-fitting, and confusion matrices.

2.3 Using the network

To use the networks in your workflow, you must add the following XML (suitably modified for your specific configuration), immediately after the LArPreProcessing algorithm in the Neutrino XML configuration.

```
<algorithm type = "LArDLVertexing">
  <TrainingMode>false</TrainingMode>
  <OutputVertexListName>NeutrinoVertices3D_Pass1</OutputVertexListName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames
      >
  <ModelFileNameU>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_U_v04_06_00.pt</ModelFileNameU>
  <ModelFileNameV>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_V_v04_06_00.pt</ModelFileNameV>
  <ModelFileNameW>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_1_W_v04_06_00.pt</ModelFileNameW>
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875
      0.59125 0.67375 0.75625 0.85 1.0</DistanceThresholds>
  <Visualise>false</Visualise>
</algorithm>

<algorithm type = "LArDLVertexing">
  <TrainingMode>false</TrainingMode>
  <Pass>2</Pass>
  <InputVertexListName>NeutrinoVertices3D_Pass1</InputVertexListName>
  <OutputVertexListName>NeutrinoVertices3D</OutputVertexListName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames
      >
  <ModelFileNameU>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_2_U_v04_06_00.pt</ModelFileNameU>
  <ModelFileNameV>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_2_V_v04_06_00.pt</ModelFileNameV>
  <ModelFileNameW>PandoraNetworkData/
      PandoraNet_Vertex_DUNEFD_HD_Accel_2_W_v04_06_00.pt</ModelFileNameW>
  <ImageWidth>128</ImageWidth>
  <ImageHeight>128</ImageHeight>
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875
      0.59125 0.67375 0.75625 0.85 1.0</DistanceThresholds>
  <Visualise>false</Visualise>
</algorithm>
```

Please following the network naming conventions. Here you can see that we have a Vertex network, for DUNE's HD far detector, in the Accel (long baseline) workflow, for the respective pass, the respective view, and using the network models released for larpandoracontent v04.06.00. The .pt files and and corresponding .pkl files should be retained to be stored in Pandora's model cache, with the .pt files being copied to the appropriate DUNE StashCache location (talk to the Pandora team).

3 Higher order vertexing

Available since v04.13.00.

3.1 Training set preparation

Generation of the training samples follows a procedure very similar to that of primary vertexing. The principal difference is that the higher order vertexing uses only a single pass. Otherwise, the mode of operation and outputs is the same. It is handled within the DlSecondaryVertexingAlgorithm, which shares much of its functionality with primary vertexing. The XML file for training can be found in `settings/development` folder of the LArReco repository. In particular, `settings/development/PandoraSettings_SecVtxTrain_DUNEFD.xml`. The current configuration is

```
<algorithm type = "LArDLSecondaryVertexing">
  <TrainingMode>true</TrainingMode>
  <TrainingOutputFileName>DUNEFD_SecVtx</TrainingOutputFileName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames>
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875 0.59125
      0.67375 0.75625 0.85 1.0</DistanceThresholds>
</algorithm>
```

3.2 Training the network

Training of the network itself uses the same architecture as the primary vertexing network, as ultimately the network architecture is identical. As above, the key difference is the single pass, rather than two passes, so you can follow the same procedure as for the primary vertexing network, making the appropriate edits to filenames, etc.

3.3 Using the network

At the time of writing, the secondary vertexing network is not part of any default workflow, but it can be used, assuming the training configuration above, by inclusion of the follow XML

```
<algorithm type = "LArDLSecondaryVertexing">
  <TrainingMode>false</TrainingMode>
  <OutputVertexListName>SecondaryVertices3D</OutputVertexListName>
  <CaloHitListNames>CaloHitListW CaloHitListU CaloHitListV</CaloHitListNames>
  <ModelFileNameU>PandoraNetworkData/
      PandoraNet_SecVertex_DUNEFD_Accel_1_U_v04_13_00.pt</ModelFileNameU>
  <ModelFileNameV>PandoraNetworkData/
      PandoraNet_SecVertex_DUNEFD_Accel_1_V_v04_13_00.pt</ModelFileNameV>
  <ModelFileNameW>PandoraNetworkData/
      PandoraNet_SecVertex_DUNEFD_Accel_1_W_v04_13_00.pt</ModelFileNameW>
  <DistanceThresholds>0. 0.00275 0.00825 0.01925 0.03575 0.05775 0.08525
      0.12375 0.15125 0.20625 0.26125 0.31625 0.37125 0.42625 0.50875 0.59125
      0.67375 0.75625 0.85 1.0</DistanceThresholds>
  <Visualise>false</Visualise>
</algorithm>
```

It is recommended to include this XML block immediately after the second pass of the primary vertexing network. Outputs are then available to subsequent algorithms by accessing the `SecondaryVertices3D` vertex list.

4 Interaction hierarchy building

Available since v04.15.00.

4.1 Training set preparation

The training set is created by the DLNeutrinoHierarchyAlgorithm. The algorithm replaces `LArNeutrinoHierarchy` and is placed between between `LArNeutrinoCreation` and `LArNeutrinoDaughterVertices` in `PandoraSettings_Neutrino_DUNEFD.xml`. The following configuration is used to train the algorithm's networks:

```
<algorithm type = "LArMLPNeutrinoHierarchy">
   <NeutrinoPfoListName>NeutrinoParticles3D</NeutrinoPfoListName>
   <PfoListNames>TrackParticles3D ShowerParticles3D</PfoListNames>
   <TrainingMode>true</TrainingMode>
   <tool type = "LArMLPCheatHierarchy" description = "
       MLPCheatHierarchyTool"/>
   <tool type = "LArMLPPrimaryHierarchy" description = "
       MLPPrimaryHierarchyTool">
     <Normalise>false</Normalise>
     <TrainingMode>true</TrainingMode>
   </tool>
   <tool type = "LArMLPLaterTierHierarchy" description = "
       MLPLaterTierHierarchyTool">
     <Normalise>false</Normalise>
     <TrainingMode>true</TrainingMode>
   </tool>
</algorithm>
```

If run in training mode, the algorithm will produce a training file, called `DLHierarchyTrainingFile.root`, which contains the following trees:

- `EventTree`,
- `PrimaryTrackTree`,
- `PrimaryShowerTree`,
- `LaterTierTrackTrackTree`, and
- `LaterTierTrackShowerTree`

The last four trees are the training trees and are flattened in order to minimise file size and I/O runtime. This means that each entry in the tree corresponds to an edge of a potential parent-child link. The `EventTree` is only used when tuning the network, where it is used to turn the flat trees into event-level trees.

4.2 Training the networks

The variables used in the algorithm's networks are orientation dependent i.e they depend on which 'endpoint' is assumed to be a particle's start/end position. This naturally leads to a two stage process when deciding whether a parent-child link is correct. In the first stage, one passes the variables of each definable edge through an 'edge model'. The edge model's output is a probability score for the following three classes: correct edge with correct orientation, correct edge but with the wrong orientation, or incorrect edge. In the second stage, the edge network outputs are concatenated together and passed through a 'classification model'. The classification model outputs a score representing the probability that the parent-child link is correct.

For tracks one can define two endpoints, and for showers we can define one endpoint (the shower region has no physical endpoint). This leads to the following workflows:

- Primary Tracks: Two edges can connect the neutrino and track particle. We have a branch model and a classification model.
- Primary Showers: One edge can connect the neutrino and shower particle. We only have a classification model.
- Track-Track links: Four edges can connect the track parent and track child particles. We have a branch model and a classification model.
- Track-Shower links: Two edges can connect the track parent and track child particles. We have a branch model and a classification model.

To train the primary models:

1. Run `PrimaryTier/WritePrimaryTierFile.ipynb` with `isTrackMode` set to `True` to obtain `/files/hierarchy_TRAIN_track.npz` and set to `False` to obtain `/files/hierarchy_TRAIN_shower.npz`.

2. Run `PrimaryTier/TrainPrimaryTierModel_tracks.ipynb` to train the primary track edge and classifier models and `PrimaryTier/TrainPrimaryTierModel_showers.ipynb` to to train the primary shower classifier model.

To train the 'later-tier' models:

1. Run `LaterTier/WriteLaterTierFile.ipynb` with `isTrackMode` set to `True` to obtain `/files/hierarchy_TRAIN_later_tier_track.npz` and set to `False` to obtain `/files/hierarchy_TRAIN_later_tier_shower.npz`.

2. Run `LaterTier/TrainLaterTierModel.ipynb` with `isTrackMode` set to `True` to train the later-tier track-track edge and classifier models.

3. Run `LaterTier/TrainLaterTierModel.ipynb` with `isTrackMode` set to `False` to train the later-tier track-shower edge and classifier models.

After these steps one should have created the following models:

- `primary_track_branch_model`
- `primary_track_classifier_model`
- `primary_shower_classifier_model`
- `track_track_branch_model`
- `track_track_classifier_model`
- `track_shower_branch_model`
- `track_shower_classifier_model`

4.3 Using the network

The primary networks are invoked in the `DLPrimaryHierarchy` tool, which is called by the `LArNeutrinoHierarchy` algorithm in its `GetPrimaryScore` function. The tool returns a score reflecting how likely the input neutrino-child particle link is to be correct. This score is then used, with reference to predetermined thresholds, to construct the primary tier.

The later-tier networks are invoked in the `DLLaterTierHierarchy` tool, which is called by the `LArNeutrinoHierarchy` algorithm in its `GetLaterTierScore` function. The tool returns a score reflecting how likely the input parent-child particle link is to be correct. This score is then used, with reference to predetermined thresholds, to construct the later-tiers.

The predetermined thresholds are determined offline using the following workflow:

1. `AddScores.ipynb` is run on the training file produced by the `LArNeutrinoHierarchy` algorithm. This notebook converts the input flat trees into event-level trees and runs the models to obtain the primary/later-tier scores for all potential links.

2. `TuneCut.ipynb` must then be run. This notebook builds the hierarchies using the same logic as implemented in the `LArNeutrinoHierarchy` algorithm. The hierarchy building proceeds in a step-wise fashion, and in each stage thresholds are used to determine whether parent-child links are made. In this notebook, metrics are produced at each stage of the hierarchy building allowing the user to identify the optimal thresholds.

3. `HierarchyMetrics.ipynb` is finally run. This notebook reports detailed hierarchy building metrics at each stage of the hierarchy building procedure.

5 Supernova neutrino and low energy background separation

Available since v04.12.00.

5.1 Training set preparation

The structure and workflow of this algorithm largely follows the DlVertexingAlgorithm, using a similar network structure, with only minor modifications to the notebooks and C++ code to produce the CSV files. Generation of the training samples is handled within the DlSNSignalAlgorithm. The `PrepareTrainingSample` function which generates a set of output CSV files for each of the views U, V and W. The XML files for training can be found in the `settings/development` folder of the LArReco repository. In particular, `PandoraSettings_DLTrain_Pass1_LowE_Backgrounds_HD.xml`, `PandoraSettings_DLTrain_Pass1_LowE_Backgrounds_VD.xml`, `PandoraSettings_DLTrain_Pass2_LowE_Backgrounds_HD.xml`, and `PandoraSettings_DLTrain_Pass2_LowE_Backgrounds_VD.xml`. This is a specific workflow for Low Energy. The signal identification is split into two passes to obtain the required reconstruction resolution. For pass 1 training, an example XML is

```
<algorithm type = "LArDLSNSignal">
    <TrainingMode>true</TrainingMode>
    <PrintOut>false</PrintOut>
    <TrainingOutputFileName>DUNEFD_VD_LowE_Pass1</TrainingOutputFileName>
    <ApplyCheatedSeparation>false</ApplyCheatedSeparation>
    <CaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
        CaloHitListNames>
    <CaloHitListName2D>SSCaloHitList2D</CaloHitListName2D>
    <InputCaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
        InputCaloHitListNames>
    <SignalListNameU>SCaloHitListU</SignalListNameU>
    <SignalListNameV>SCaloHitListV</SignalListNameV>
    <SignalListNameW>SCaloHitListW</SignalListNameW>
    <SignalListName2D>SCaloHitList2D</SignalListName2D>
    <BackgroundListName>BBCaloHitList</BackgroundListName>
    <ZoomCaloHitListU>SCaloHitListU</ZoomCaloHitListU>
    <ZoomCaloHitListV>SCaloHitListV</ZoomCaloHitListV>
    <ZoomCaloHitListW>SCaloHitListW</ZoomCaloHitListW>
    <Visualise>false</Visualise>
    <VisualiseZoom>false</VisualiseZoom>
</algorithm>
```

This will generate output comma-separated value (CSV) files called `DUNEFD_VD_LowE_Pass1_CaloHitListU.csv`, `DUNEFD_VD_LowE_Pass1_CaloHitListV.csv`, and `DUNEFD_VD_LowE_Pass1_CaloHitListW.csv`. Unlike the vertex network, there are no thresholds here as classification is based on parent particle rather than distance.

For this low energy training, there are two classes in Pass 1 and three classes in Pass 2. These are for (background, signal) and (background, photon, electron) respectively

For pass 2 training, the XML structure runs the pass 1 network in inference mode (that is repeats the above block, with some modifications as noted below, and then runs a second pass, as follows

```
<algorithm type = "LArDLSNSignal">
    <TrainingMode>false</TrainingMode>
    ...
    <ModelFileNameU>PandoraNetworkData/
        PandoraNet_SNSignalTag_DUNEFD_VD_1_U_v04_12_00.pt</ModelFileNameU>
    <ModelFileNameV>PandoraNetworkData/
        PandoraNet_SNSignalTag_DUNEFD_VD_1_V_v04_12_00.pt</ModelFileNameV>
    <ModelFileNameW>PandoraNetworkData/
        PandoraNet_SNSignalTag_DUNEFD_VD_1_W_v04_12_00.pt</ModelFileNameW>
</algorithm>

<algorithm type = "LArDLSNSignal">
    <TrainingMode>true</TrainingMode>
```

```
        <TrainingOutputFileName>DUNEFD_VD_LowE_Pass2</TrainingOutputFileName>
        <PrintOut>false</PrintOut>
        <Pass>2</Pass>
        <ImageHeight>128</ImageHeight>
        <ImageWidth>128</ImageWidth>
        <CaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
            CaloHitListNames>
        <CaloHitListName2D>SSCaloHitList2D</CaloHitListName2D>
        <InputCaloHitListNames>SCaloHitListW SCaloHitListU SCaloHitListV</
            InputCaloHitListNames>
        <SignalListNameU>CaloHitListU</SignalListNameU>
        <SignalListNameV>CaloHitListV</SignalListNameV>
        <SignalListNameW>CaloHitListW</SignalListNameW>
        <SignalListName2D>CaloHitList2D</SignalListName2D>
        <BackgroundListName>BCaloHitList</BackgroundListName>
        <ModelFileNameU>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_U_v04_12_00.pt</ModelFileNameU>
        <ModelFileNameV>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_V_v04_12_00.pt</ModelFileNameV>
        <ModelFileNameW>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_W_v04_12_00.pt</ModelFileNameW>
        <Visualise>false</Visualise>
    </algorithm>
```

The key differences here are the requirement to run the trained pass 1 network first, in inference, rather than training mode, provide the name of the output signal list, and specify the names of the trained models from pass 1 for each of the respective views. This is important because training on a true signal in pass 2 would cause the signal to be located in preferred positions due to the framing of the image and potentially bias the network. Also notice that we specify the pass in the second instance, and also specify image dimensions in pass 2, which is half of the default dimension, for computational efficiency.

5.2 Training the network

Python Notebooks are provided for training the network. These notebooks are self-documenting, so only a brief overview will be given here. You will need the CSV files created during training set preparation, starting with the pass 1 files, given the dependence of pass 2 on pass 1 training. The `make_images.ipynb` notebook can be run from start to finish, modulo any edits that may be needed according to any customisations you applied (as described in the notebook).

The `network.ipynb` notebook can then run on the resultant compressed numpy arrays from the image generation step. You'll need GPU resources for this, and ensure your files are available on the GPU node (or a suitable high speed, highly parallel I/O system). As with the previous notebook, potential edits according to customisation and which view you are running on will need to be made, but otherwise you can run the notebook from start to finish.

This will produce the trained networks for every epoch, and the TorchScript (.pt) network that will actually run in Pandora, along with various metrics showing training and validation loss and accuracy, to check for over-fitting, and confusion matrices.

5.3 Using the network

To use the networks in your workflow, you must add the following XML (suitably modified for your specific configuration), immediately after the LArDlVertexing algorithm in the Neutrino XML configuration.

```
<algorithm type = "LArDLSNSignal">
        <TrainingMode>false</TrainingMode>
        <PrintOut>false</PrintOut>
        <TrainingOutputFileName>LowE_P1</TrainingOutputFileName>
        <ApplyCheatedSeparation>false</ApplyCheatedSeparation>
```

```
        <CaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
            CaloHitListNames>
        <CaloHitListName2D>SSCaloHitList2D</CaloHitListName2D>
        <InputCaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
            InputCaloHitListNames>
        <SignalListNameU>SCaloHitListU</SignalListNameU>
        <SignalListNameV>SCaloHitListV</SignalListNameV>
        <SignalListNameW>SCaloHitListW</SignalListNameW>
        <SignalListName2D>SCaloHitList2D</SignalListName2D>
        <BackgroundListName>BBCaloHitList</BackgroundListName>
        <ZoomCaloHitListU>SCaloHitListU</ZoomCaloHitListU>
        <ZoomCaloHitListV>SCaloHitListV</ZoomCaloHitListV>
        <ZoomCaloHitListW>SCaloHitListW</ZoomCaloHitListW>
        <ModelFileNameU>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_1_U_v04_12_00.pt</ModelFileNameU>
        <ModelFileNameV>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_1_V_v04_12_00.pt</ModelFileNameV>
        <ModelFileNameW>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_1_W_v04_12_00.pt</ModelFileNameW>
        <Visualise>false</Visualise>
        <VisualiseZoom>false</VisualiseZoom>
    </algorithm>

    <algorithm type = "LArDLSNSignal">
        <TrainingMode>false</TrainingMode>
        <TrainingOutputFileName>LowE_P2</TrainingOutputFileName>
        <PrintOut>false</PrintOut>
        <Pass>2</Pass>
        <ImageHeight>128</ImageHeight>
        <ImageWidth>128</ImageWidth>
        <CaloHitListNames>SSCaloHitListW SSCaloHitListU SSCaloHitListV</
            CaloHitListNames>
        <CaloHitListName2D>SSCaloHitList2D</CaloHitListName2D>
        <InputCaloHitListNames>SCaloHitListW SCaloHitListU SCaloHitListV</
            InputCaloHitListNames>
        <SignalListNameU>CaloHitListU</SignalListNameU>
        <SignalListNameV>CaloHitListV</SignalListNameV>
        <SignalListNameW>CaloHitListW</SignalListNameW>
        <SignalListName2D>CaloHitList2D</SignalListName2D>
        <BackgroundListName>BCaloHitList</BackgroundListName>
        <ModelFileNameU>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_U_v04_12_00.pt</ModelFileNameU>
        <ModelFileNameV>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_V_v04_12_00.pt</ModelFileNameV>
        <ModelFileNameW>PandoraNetworkData/
            PandoraNet_SNSignalTag_DUNEFD_VD_2_W_v04_12_00.pt</ModelFileNameW>
        <Visualise>false</Visualise>
    </algorithm>
```

Please following the network naming conventions. Here you can see that we have a signal network, for DUNE's VD far detector, in the Low Energy workflow, for the respective pass, the respective view, and using the network models released for larpandoracontent v04.12.00. The .pt files and and corresponding .pkl files should be retained to be stored in Pandora's model cache, with the .pt files being copied to the appropriate DUNE StashCache location (talk to the Pandora team).